



STACK

Service Desk AI Solution

Project Documentation

Version: 1.0 — Initial Release

Date: May 27, 2025

Project Lead Sanjay Deshpande

Project Guide Dattatraya Girawale

Lead Developer Nikhil Rokade

Chatbot Developer Priyanka Chougule

Classification: Internal Development

1. Executive Summary

The STACK Service Desk AI Solution is an intelligent, end-to-end IT service desk automation platform designed to reduce manual effort, accelerate ticket resolution, and improve overall IT support quality. The system integrates with Freshservice (ITSM), the Microsoft ecosystem (SharePoint, O365, Graph API), and Google Workspace to automatically triage, process, and resolve common IT service requests without human intervention.

The solution is built on two primary delivery channels:

- React.js Web Dashboard — for IT admins and managers to monitor, configure, and report
- Google Chat Bot — for end users to raise and track requests conversationally within their existing workspace

The core intelligence is powered by Azure OpenAI GPT-4o, orchestrated through LangChain Agents and a RAG (Retrieval-Augmented Generation) pipeline that reads from your internal SOP library to make accurate, context-aware resolution decisions.

2. Problem Statement

IT service desks today face the following recurring challenges:

- High volumes of repetitive, low-complexity tickets (access requests, license provisioning, DL updates, basic troubleshooting) consuming valuable agent time
- Slow resolution times caused by manual, step-by-step processing of each ticket
- Inconsistent resolution quality depending on agent availability and experience
- No single visibility layer for logs, performance metrics, and management reporting
- End users have no real-time self-service channel for common IT issues
- No proactive SLA monitoring — breaches are noticed only after they occur

The STACK Service Desk AI Solution directly addresses all of the above by automating resolution, standardising quality through SOP-grounded AI decisions, and providing comprehensive real-time dashboards.

3. Project Scope

3.1 In-Scope Use Cases

Use Case ID	Category	Sub-Type	Description
UC-01a	SharePoint Tickets	Access Related	Granting/revoking site access, permission changes
UC-01b	SharePoint Tickets	Admin Related	Site creation, settings changes, ownership transfers
UC-02	License Requests	BlueBeam / Adobe / O365	License assignment and revocation for all three platforms
UC-03	DL Updates	O365 Distribution Lists	Add/remove members, create and modify distribution lists
UC-04	Windows Troubleshooting	Basic Issues	Password reset, printer, software install/uninstall, common errors

3.2 Out of Scope (Phase 1)

- Hardware replacement workflows
- Network infrastructure tickets
- Server-level incident management
- Tickets requiring physical on-site intervention

4. Solution Architecture

4.1 Architecture Diagram

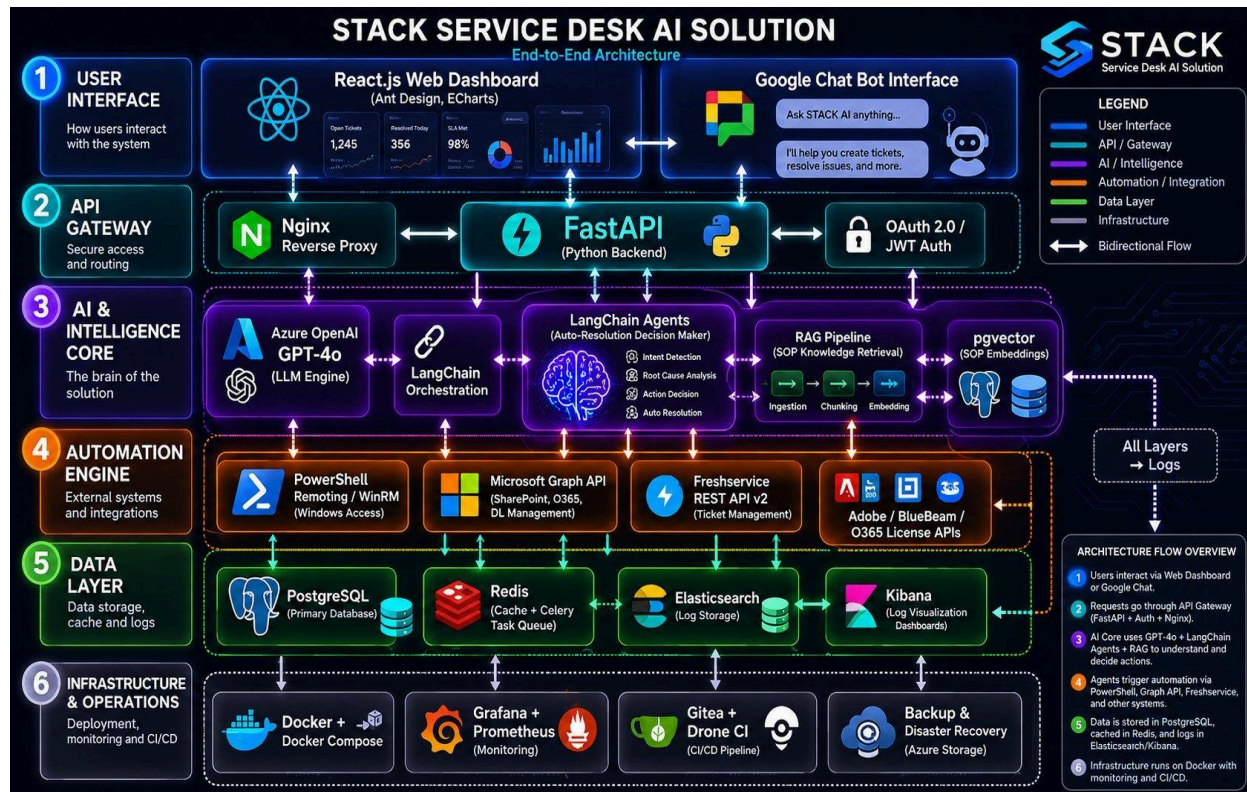


Figure 1 — STACK Service Desk AI Solution: End-to-End Architecture

4.2 Architecture Layers Overview

Layer	Name	Key Components
1	User Interface	React.js Web Dashboard (Ant Design, ECharts), Google Chat Bot
2	API Gateway	FastAPI (Python), Nginx Reverse Proxy, OAuth 2.0 / JWT Auth
3	AI & Intelligence Core	Azure OpenAI GPT-4o, LangChain, RAG Pipeline, pgvector, LangChain Agents
4	Automation Engine	PowerShell/WinRM, Microsoft Graph API, Freshservice REST API v2, License APIs
5	Data Layer	PostgreSQL, Redis, Elasticsearch, Kibana
6	Infrastructure & Operations	Docker + Compose, Grafana + Prometheus, Gitea + Drone CI, Backup & DR

4.3 End-to-End Ticket Flow

The following describes the complete journey of a ticket from user submission to resolution or escalation:

Step 1: Ticket Ingestion — User raises a ticket via Freshservice portal, the React Web Dashboard, or the Google Chat Bot. All channels feed into the FastAPI backend through Nginx.

Step 2: Authentication & Routing — The request is authenticated via OAuth 2.0 / JWT. FastAPI identifies the ticket type and routes it to the AI & Intelligence Core.

Step 3: AI Triage — LangChain Agents perform: Intent Detection, Root Cause Analysis, SOP Retrieval via RAG Pipeline, and Action Decision (auto-resolve or escalate).

Step 4: Confidence Scoring — AI calculates a confidence score (0-100%) before proceeding. Score above 85% triggers auto-resolution. Score below 60% triggers human escalation.

Step 5: Auto Resolution — If confident: SharePoint tickets via Graph API, License requests via vendor APIs, DL updates via Graph API, Windows troubleshooting via PowerShell/WinRM.

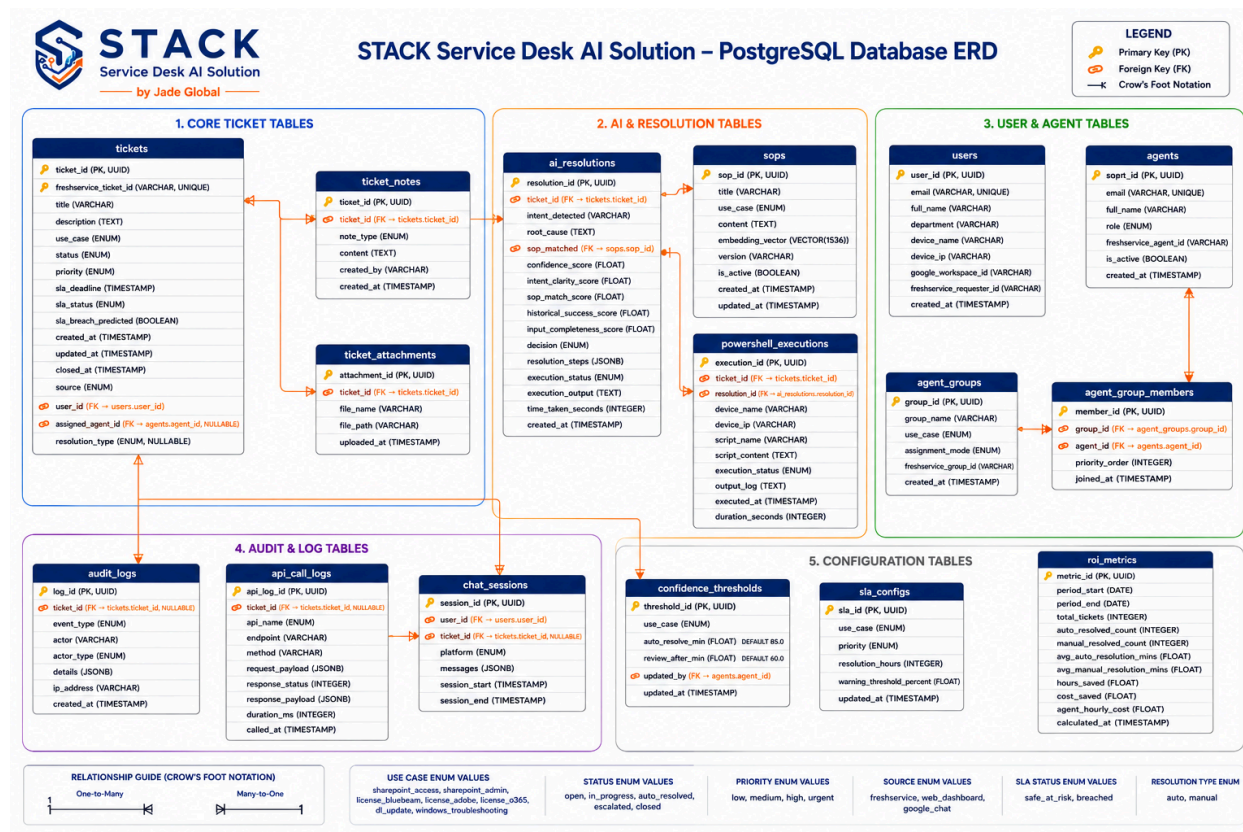
Step 6: Ticket Update — Freshservice REST API v2 is called to update ticket status, add AI-generated resolution notes, and close the ticket.

Step 7: Logging & Reporting — Every action at every step is logged to Elasticsearch. React Dashboard and Kibana surface these logs as real-time reports and dashboards.

5. Technology Stack

Layer	Technology	Purpose
Frontend	React.js (Vite) + Ant Design	Web dashboard UI
Charts	Apache ECharts / Recharts	Reporting & dashboards
Backend	FastAPI (Python)	Core API engine
Task Queue	Celery + Redis	Background automation jobs
Auth	OAuth 2.0 + JWT	Google Workspace SSO
LLM Engine	Azure OpenAI GPT-4o	AI reasoning & decisions
AI Orchestration	LangChain	Agent workflows & RAG chains
Vector Store	pgvector (PostgreSQL)	SOP embeddings & semantic retrieval
Primary DB	PostgreSQL	Tickets, logs, users, configs
Cache	Redis	Session cache, Celery task broker
Log Storage	Elasticsearch	All system & resolution logs
Log Visualization	Kibana	Log search and dashboards
Remote Execution	PowerShell Remoting / WinRM	Windows device auto-resolution
MS Integration	Microsoft Graph API	SharePoint, O365, DL actions
ITSM	Freshservice REST API v2	Ticket management & updates
Chatbot	Google Chat API (Python)	Conversational user interface
Containers	Docker + Docker Compose	On-premise deployment
Monitoring	Grafana + Prometheus	System health monitoring
CI/CD	Gitea + Drone CI	Self-hosted version control & pipeline
Backup & DR	Azure Blob Storage	Disaster recovery

6. Database ERD



6.1 Table Groups & Purpose

Group 1 — Core Ticket Tables

These are the heart of the system. Every ticket raised through any channel (Freshservice, Web Dashboard, Google Chat Bot) is stored and tracked here.

Table	Purpose
tickets	Master ticket record — stores every ticket with its use case, status, priority, SLA deadline, SLA risk status, source channel, assigned user, assigned agent, and resolution type
ticket_notes	All notes attached to a ticket — AI-generated context notes, resolution summaries, human agent notes, and system-generated notes
ticket_attachments	Any files or screenshots attached to a ticket by the user or agent

Group 2 — AI & Resolution Tables These tables capture everything the AI does — every decision, every score, every SOP it matched, and every PowerShell command it executed.

Table	Purpose
ai_resolutions	Full AI triage record per ticket — intent detected, root cause, SOP matched, confidence score (with all 4 sub-scores), decision taken, resolution steps executed, and outcome
sops	SOP knowledge base — stores all Standard Operating Procedures with their content and pgvector embeddings (VECTOR 1536) for semantic retrieval by the RAG pipeline
powershell_executions	Full log of every PowerShell/WinRM remote execution — device details, script name, script content, output log, status, and duration

Group 3 — User & Agent Tables These tables manage everyone who interacts with the system — end users who raise tickets, agents who resolve escalations, the groups they belong to, and how escalations are assigned.

Table	Purpose
users	End user profiles — email, department, device name, device IP, Google Workspace ID, and Freshservice requester ID
agents	IT agent profiles — email, role (Admin / Agent / Read-Only), Freshservice agent ID, and active status
agent_groups	The 4 escalation groups with their use case mapping and assignment mode (Round-Robin / First Available / Priority)
agent_group_members	Junction table linking agents to their groups with a configurable priority order

The 4 agent groups configured in the system are:

Group	Use Case	Members
SharePoint Support	SharePoint Tickets	Bhaskar, Neha, Madhura, Yogesh, Datta
License Management	License Requests	Neha, Madhura, Datta
DL Management	DL Updates	Neha, Madhura, Datta
L2 Windows Support	Windows Troubleshooting	Ajay, Ankit, Hadi, Shameem, Datta

Group 4 — Audit & Log Tables Every action taken in the system — by a user, by the AI, by an agent, or by an automated process — is recorded here. These tables feed both Elasticsearch and the Kibana log dashboards.

Table	Purpose
audit_logs	Complete audit trail of all system events — ticket created, AI triage, auto-resolved, escalated, human resolved, SLA breached, PowerShell executed, API called, user notified
api_call_logs	Full log of every external API call made by the system — Freshservice, Microsoft Graph API, Adobe, BlueBeam, O365, Azure OpenAI, PowerShell — with request/response payloads, status codes, and duration
chat_sessions	Full conversation history for every Google Chat Bot and Web Dashboard chat session, stored as JSONB

Group 5 — Configuration Tables These tables allow admins to configure system behaviour without touching code. All thresholds, SLA rules, and ROI parameters are stored here and editable from the Admin Dashboard.

Table	Purpose
confidence_thresholds	Per-use-case confidence score thresholds — configurable auto-resolve minimum (default 85%) and review-after minimum (default 60%)
sla_configs	SLA resolution time targets per use case and priority level, with configurable warning thresholds
roi_metrics	Aggregated ROI data per period — total tickets, auto-resolved count, hours saved, cost saved, and agent hourly cost — used to power the ROI Dashboard

6.2 Key Design Decisions

- **UUID Primary Keys**
All tables use UUID as the primary key instead of auto-increment integers. This prevents enumeration attacks, supports distributed inserts, and makes the database portable across environments.
 - **JSONB Columns**
Used for resolution_steps, details, messages, request_payload, and response_payload. JSONB is natively indexed in PostgreSQL, making it fast to query while allowing flexible nested structures that don't need a fixed schema.
 - **pgvector Extension**
The sops table stores a VECTOR(1536) column for each SOP's Azure OpenAI text-embedding-3 embedding. This enables the RAG pipeline to perform cosine similarity searches to find the most relevant SOP for any incoming ticket — entirely within PostgreSQL, with no separate vector database required.
 - **ENUM Types**
All categorical fields (status, priority, use_case, source, resolution_type, etc.) use PostgreSQL native ENUMs. This enforces data integrity at the database level and makes queries faster and safer.
 - **NULLABLE Foreign Keys**
Fields like assigned_agent_id (tickets) and ticket_id (audit_logs, api_call_logs) are nullable because some log entries and configurations exist independently of a specific ticket.
 - **Datta as Cross-Functional Fallback**
Datta appears as a member in all 4 agent groups (agent_group_members). In the system, if all primary group members are unavailable, the assignment logic falls back to Datta automatically. This is configurable via the priority_order column in agent_group_members.
-

6.3 ENUM Reference

ENUM	Values
use_case	sharepoint_access, sharepoint_admin, license_bluebeam, license_adobe, license_o365, dl_update, windows_troubleshooting
status	open, in_progress, auto_resolved, escalated, closed
priority	low, medium, high, urgent
source	freshservice, web_dashboard, google_chat
sla_status	safe, at_risk, breached
resolution_type	auto, manual
assignment_mode	round_robin, first_available, priority
actor_type	user, ai, agent, system
note_type	ai_context, resolution_summary, human_note, system_note
execution_status	success, failed, timeout, partial
decision	auto_resolve, review_after, escalate

7. AI Intelligence Features

7.1 Confidence Score — How It Works

Before executing any auto-resolution, the AI calculates a Confidence Score (0-100%) based on four weighted factors:

Factor	Weight	What It Measures
Intent Clarity	30%	How clearly the AI understood the user's request. Ambiguous language lowers this score.
SOP Match	35%	How closely the RAG pipeline matched a relevant SOP. Higher similarity = higher score.
Historical Success	25%	Whether this exact ticket type has been successfully auto-resolved before (from PostgreSQL logs).
Input Completeness	10%	Whether the ticket contains all required fields (user ID, device name, software name, etc.).

Score Thresholds:

- 85-100% — Auto-resolve immediately, no human review required
- 60-84% — Auto-resolve but flag for human review post-resolution
- Below 60% — Escalate to human agent; AI adds full context notes to the ticket

Thresholds are configurable per use case from the Admin Dashboard. Higher-risk use cases (e.g. SharePoint Admin) can be set to require stricter confidence thresholds than lower-risk ones (e.g. basic troubleshooting).

7.2 Human Escalation — How It Works

When the AI cannot resolve a ticket confidently, the following escalation process is triggered automatically:

1. AI attempts resolution and either fails or scores below the configured confidence threshold.
2. AI writes a detailed internal note on the Freshservice ticket — documenting what was attempted, what failed, and what it recommends the human agent do next.
3. The ticket is automatically reassigned via the Freshservice API to the appropriate agent group based on the use case.
4. The end user is notified via Google Chat Bot: "Your request requires attention from our support team. A human agent has been assigned and will reach out shortly."
5. The assigned human agent picks up the ticket in Freshservice with full AI-generated context already documented — so they never start from scratch.

7.3 Escalation Assignment Matrix

Use Case	Freshservice Group	Members	Assignment Mode
SharePoint Tickets	SharePoint Support Group	Bhaskar, Neha, Madhura, Yogesh, Datta	Admin Configurable
License Requests	License Management Group	Neha, Madhura, Datta	Admin Configurable
DL Updates	DL Management Group	Neha, Madhura, Datta	Admin Configurable
Windows Troubleshooting	L2 Windows Support Group	Ajay, Ankit, Hadi, Shameem, Datta	Admin Configurable

Note: Datta is a member across all 4 groups and serves as the cross-functional escalation fallback. If all primary agents in a group are unavailable, Datta receives the ticket. Assignment mode (Round-Robin, First Available, Priority-Based) is configurable per group from the Admin Dashboard.

7.4 SLA Breach Prediction

The AI monitors every open ticket in real time using a Celery background job. It predicts SLA breach risk by evaluating:

- Current ticket age vs SLA deadline
- Ticket complexity score — based on historical resolution times for similar ticket types
- Current queue load — number of tickets ahead in the same category
- Time of day and agent availability — fewer active agents = higher predicted resolution time

Example:

Scenario: A SharePoint access ticket is raised at 4:45 PM Friday. SLA = 4 hours. AI detects: 3 similar tickets in queue, agents offline, predicted resolution time = 5.2 hours. System immediately flags as HIGH breach risk and either triggers auto-resolution or escalates with URGENT priority.

The dashboard displays a live SLA Risk column for every open ticket:

- Green — Safe (resolution well within SLA)
- Yellow — At Risk (resolution time approaching SLA deadline)
- Red — Breach Imminent (predicted to breach; immediate action required)

SLA risk statuses are refreshed every few minutes via Celery background jobs and visible in real time on the React Dashboard.

8. Key Features

8.1 Core Automation Features

- **Auto Ticket Resolution** — AI reads the matched SOP and executes resolution steps autonomously for all 4 use cases
- **Device Access for Automation** — PowerShell/WinRM connects to user's Windows machine for hands-free Windows troubleshooting
- **Multi-Channel Intake** — Freshservice portal, React Dashboard, and Google Chat Bot all feed into one unified resolution engine
- **Human Escalation with AI Context** — graceful escalation with full AI-generated notes so agents never start from scratch
- **SOP-Grounded Decisions** — every AI action is backed by your actual internal SOP documents via the RAG pipeline

8.2 AI-Powered Intelligence Features

- **Intent Classification** — understands what the user wants from natural language input
- **Root Cause Analysis** — identifies the underlying issue beyond the surface-level request
- **Confidence Scoring** — AI assigns a weighted confidence score before auto-resolving; low confidence triggers escalation
- **SLA Breach Prediction** — proactively flags tickets at risk of breaching SLA before it happens
- **Ticket Summarization** — auto-generates concise resolution summaries for all closed tickets
- **Trend Analysis** — identifies recurring ticket patterns and surfaces actionable recommendations
- **Anomaly Detection** — flags unusual spikes in ticket volumes (potential incidents or outages)
- **Predictive Escalation** — predicts overload scenarios before they affect SLA compliance

8.3 Dashboard & Reporting Features

- **Live ticket queue** with status, priority, assigned agent, and AI confidence score
- **Resolution rate by use case** — SharePoint, License, DL Updates, Windows Troubleshooting
- **Average resolution time** — AI auto-resolved vs manually handled side-by-side
- **SLA breach tracker** with 3-color risk indicator (Green / Yellow / Red)
- **Daily, weekly, and monthly exportable reports**
- **Full audit log per ticket** — every AI step, every API call, every outcome timestamped
- **Agent performance reports** for human-handled escalations

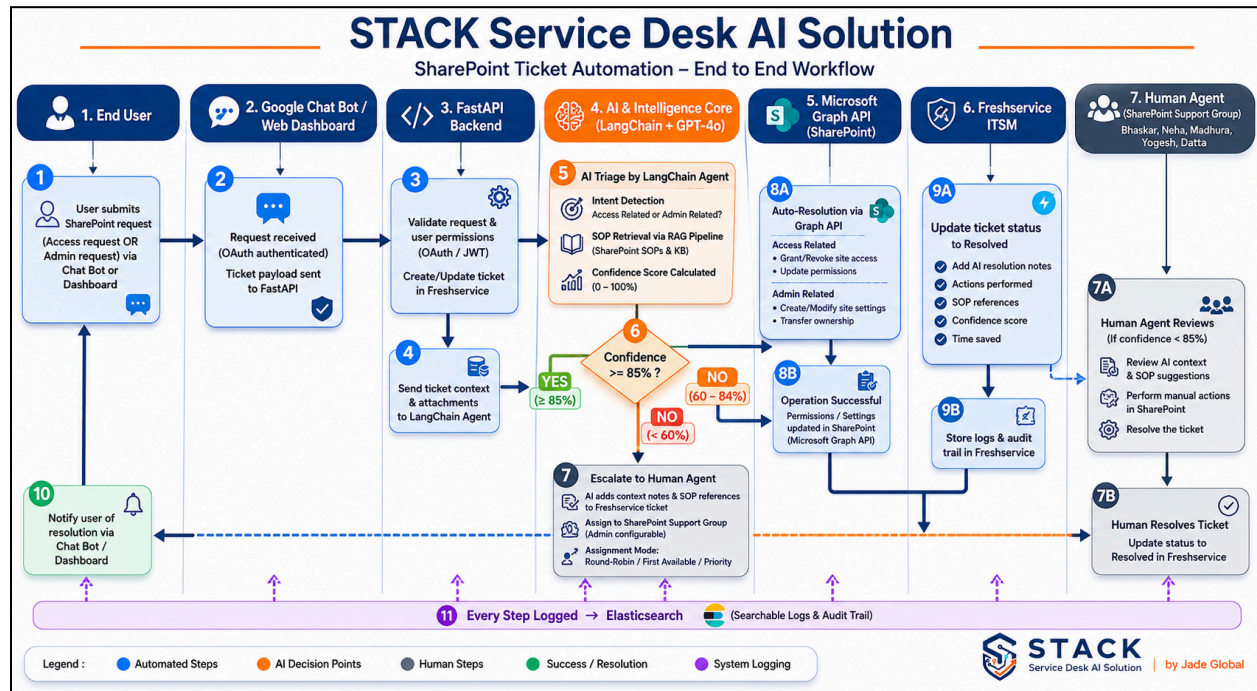
8.4 ROI Dashboard

The ROI Dashboard provides management with a real-time business case showing exactly how much time and money the system is saving:

Metric	How It Is Calculated
Tickets Auto-Resolved	Count of AI-closed tickets vs total tickets received
Hours Saved	Auto-resolved tickets x average manual resolution time (configurable)
Cost Saved	Hours saved x average agent hourly cost (configurable in settings)
Agent Capacity Freed	Percentage of agent time freed up by AI automation
Cost Per Ticket (AI vs Human)	Side-by-side cost comparison between AI and human resolution
Monthly / Quarterly ROI Trend	Chart showing ROI growth over time as AI resolves more tickets
Payback Period Indicator	Estimated time to recover the project development investment

9. Use case WorkFlow:

9.1 Sharepoint tickets



1. Purpose of the Workflow

The purpose of this workflow is to:

- Automate SharePoint access and administration requests
- Reduce ticket resolution time
- Minimize repetitive manual operations
- Improve SLA compliance
- Provide intelligent AI-driven ticket triage
- Escalate low-confidence cases to human agents

2. Workflow Participants (Swimlanes)

Swimlane	Description
End User	Employee submitting SharePoint request
Google Chat Bot / Web Dashboard	User interaction interface
FastAPI Backend	Core API orchestration layer
AI & Intelligence Core	LangChain + GPT-4o reasoning engine
Microsoft Graph API	SharePoint automation execution layer
Freshservice ITSM	Ticket lifecycle management
Human Agent	Manual resolution support team

3. End-to-End Workflow Description

Step 1 — User Request Submission

Actor: End User

The user submits a SharePoint-related request using:

- Google Chat Bot
- STACK Web Dashboard

Supported Request Types

- SharePoint Access Request
- SharePoint Admin Request
- Site Permission Changes
- Ownership Transfer
- Site Configuration Updates

Inputs Captured

- User identity
- Site URL
- Request type
- Business justification
- Priority

Step 2 — Authentication & Request Intake

Actor: Google Chat Bot / Web Dashboard

The system authenticates the user using:

- OAuth 2.0
- JWT Tokens
- Enterprise SSO integration

After validation:

- Request payload is generated
- Metadata is attached
- Ticket request is forwarded to FastAPI backend

Step 3 — Backend Processing

Actor: FastAPI Backend

- The FastAPI orchestration layer performs:
- Request validation
- Payload normalization
- Session tracking
- Audit logging
- Ticket creation/update in Freshservice

Backend Responsibilities

- API Gateway routing
- Security validation
- Request enrichment
- AI orchestration trigger

Step 4 — AI-Powered Ticket Triage

Actor: AI & Intelligence Core

Technology: LangChain + Azure OpenAI GPT-4o

The ticket is processed by LangChain agents for intelligent analysis.

AI Processing Activities

4.1 Intent Detection

- The AI determines whether the request is:
- Access Related
- Administration Related

4.2 SOP Retrieval (RAG Pipeline)

- Relevant SOPs and knowledge documents are retrieved using:
- Vector embeddings
- pgvector similarity search
- Retrieval-Augmented Generation (RAG)

4.3 Confidence Scoring

- The AI calculates a confidence score (0–100%) based on:
- SOP match quality
- Intent clarity
- Historical patterns
- Policy validation

4. AI Decision Logic

Confidence Score	Action
≥ 85%	Fully automated resolution
60% – 84%	Auto-resolution with review flag
< 60%	Escalate to human support

5. Automated Resolution Flow

5.1 Access-Related Requests

Actor: Microsoft Graph API

Automated actions include:

- Grant SharePoint access
- Revoke access
- Update site permissions
- Add/remove users from groups

5.2 Administration-Related Requests

Actor: Microsoft Graph API

Automated actions include:

- Create or modify SharePoint sites
- Update site settings
- Transfer ownership
- Configure permissions

6. Ticket Update & Resolution

Freshservice ITSM Updates

Actor: Freshservice ITSM

After successful automation:

- Ticket status updated to Resolved
- AI-generated notes attached
- Confidence score recorded
- Actions logged
- Audit trail maintained

7. User Notification

The user receives resolution updates through:

- Google Chat Bot
- Web Dashboard notifications
- Optional email integration

Notification Includes

- Resolution status
- Actions performed
- Approval information
- Ticket reference number

8. Human Escalation Workflow

If AI confidence is below threshold:

- Ticket is escalated to SharePoint Support Group
- AI-generated context is attached
- SOP recommendations are included

Support Team

- Bhaskar
- Neha
- Madhura
- Yogesh
- Datta

Assignment Modes

- Round-Robin
- First Available
- Priority-Based

9. Human Resolution Process

The assigned support engineer:

- Reviews AI recommendations
- Performs manual SharePoint operations
- Updates Freshservice ticket
- Resolves the request

The user is then notified of resolution completion.

10. Logging & Observability

All workflow stages continuously send logs and telemetry to Elasticsearch.

Logged Events

- User requests
- API calls
- AI decisions
- Confidence scores
- Graph API operations
- Human interventions
- Audit activities

11. Monitoring & Analytics

The system supports:

- SLA tracking
- AI confidence analytics
- Automation success rates
- Ticket trend analysis
- Failure diagnostics

Visualization tools may include:

- Kibana
- Grafana
- ECharts dashboards

12. Security & Compliance

Security Controls

- OAuth 2.0 Authentication
- JWT Authorization
- Role-Based Access Control (RBAC)
- API Gateway protection
- Secure Graph API scopes

Compliance Features

- Full audit trails
- AI decision traceability
- Ticket history retention
- Activity logging

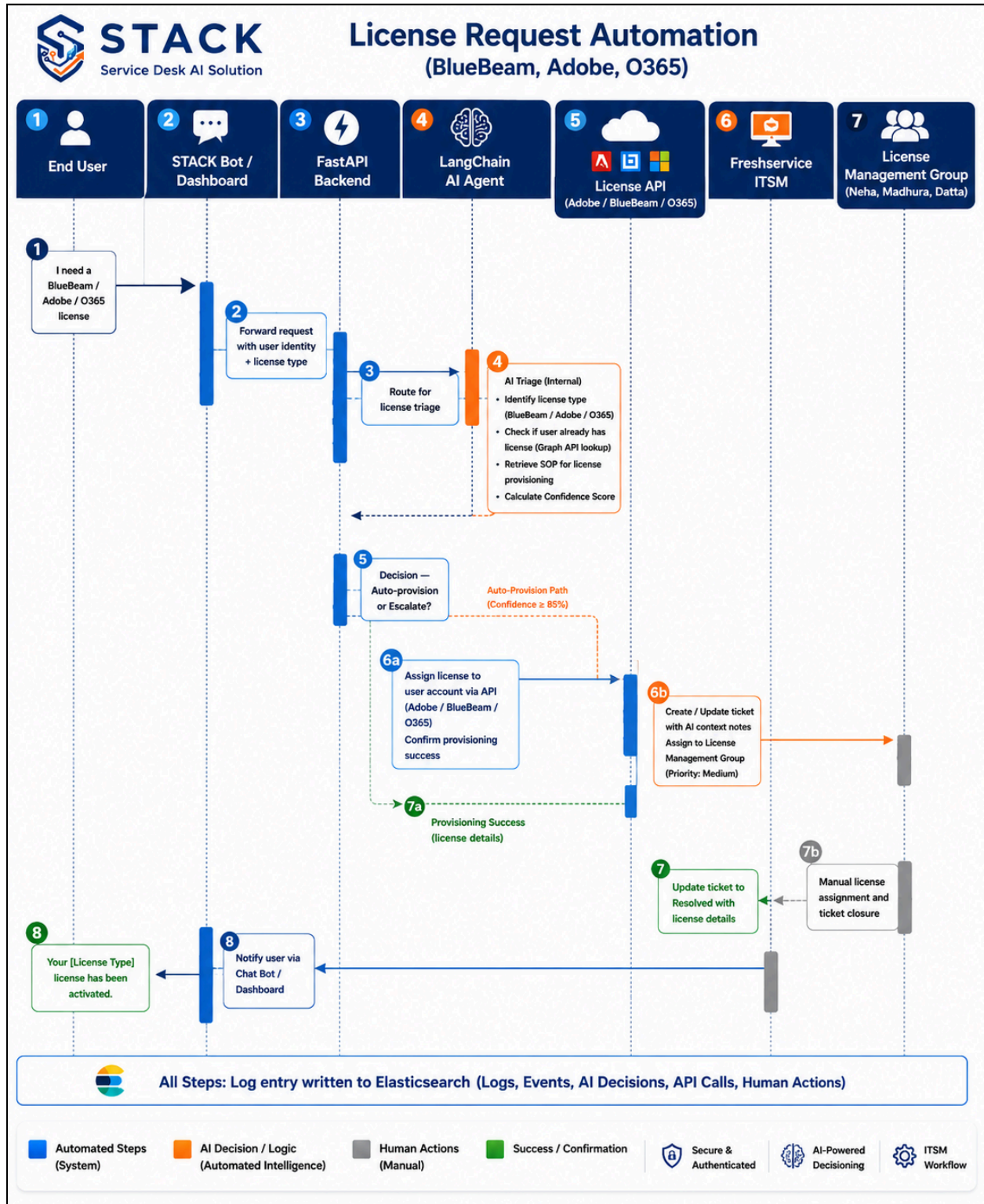
13. Business Benefits

Benefit	Impact
Faster Resolution	Reduced MTTR
AI Automation	Lower manual effort
Intelligent Routing	Improved support efficiency
Human-in-the-loop	Controlled risk
Standardized SOP execution	Higher consistency
Real-time visibility	Better operational governance

14. Conclusion

The STACK Service Desk AI Solution provides a scalable, enterprise-grade AI automation framework for SharePoint service management. By combining conversational AI, intelligent orchestration, automated API execution, and human escalation capabilities, the platform significantly improves operational efficiency while maintaining governance, security, and auditability.

9.2 License requests BlueBeam, Adobe, O365



1. Overview

The **License Request Automation** module within the **STACK Service Desk AI Solution** automates the end-to-end lifecycle of software license provisioning requests for enterprise applications such as:

- BlueBeam
- Adobe Creative Cloud
- Microsoft O365

The solution leverages:

- AI-powered ticket triage using GPT-4o and LangChain
- Automated provisioning APIs
- ITSM workflow integration via Freshservice
- Human escalation workflows for low-confidence or exception cases

The objective is to reduce manual IT effort, accelerate fulfillment time, improve auditability, and provide a seamless self-service experience for end users.

2. Business Problem

Traditional license provisioning workflows involve:

- Manual ticket triage
- Delays in identifying required license types
- Repetitive administrative effort
- Inconsistent SOP execution
- Lack of intelligent automation
- Delayed response times for employees

These issues increase operational overhead and negatively impact employee productivity.

3. Solution Objectives

The STACK AI-driven automation workflow is designed to:

- Automate software license provisioning
- Reduce IT ticket resolution time
- Minimize manual intervention
- Ensure SOP-based execution
- Improve user experience via conversational interfaces
- Enable intelligent escalation handling
- Maintain full audit and logging visibility

4. Key Features

AI-Powered Ticket Understanding

- GPT-4o-based intent recognition
- Automatic identification of requested license type
- Context-aware request analysis

Intelligent Decision Engine

- Confidence score calculation
- Auto-provisioning for high-confidence requests
- Human escalation for uncertain scenarios

Automated License Provisioning

Supports automated provisioning for:

- BlueBeam
- Adobe
- O365

Multi-Channel User Experience

Users can submit requests through:

- Google Chat Bot
- STACK Web Dashboard

ITSM Integration

- Freshservice ticket creation and updates
- Automatic resolution notes
- SLA tracking support

Human-in-the-Loop Support

Low-confidence or policy-sensitive requests are routed to:

- License Management Group
 - Neha
 - Madhura
 - Datta

5. System Components

Component	Purpose
STACK Bot / Dashboard	User interaction layer
FastAPI Backend	API orchestration and workflow routing
LangChain AI Agent	AI-based ticket triage and decision making
GPT-4o	Natural language understanding
License APIs	Provisioning licenses
Freshservice ITSM	Ticket lifecycle management
Elasticsearch	Logging and audit trail

6. Workflow Description

Step 1 — User Request Submission

The end user submits a request such as:

“I need an Adobe license”

via:

- Google Chat Bot
- STACK Web Dashboard

Step 2 — Request Authentication & Routing

The STACK frontend forwards:

- User identity
- Requested license type
- Session context

to the FastAPI backend after OAuth/JWT validation.

Step 3 — AI-Based Ticket Triage

The FastAPI backend sends the request to the LangChain AI Agent.

The AI engine performs:

- License type identification
- Existing license verification
- SOP retrieval through RAG
- Confidence score generation

Step 4 — Intelligent Decision Making

- **High Confidence ($\geq 85\%$)**
The request proceeds to automatic provisioning.
- **Medium Confidence (60–84%)**
Provisioning may proceed with monitoring/review tagging.
- **Low Confidence ($< 60\%$)**
The ticket is escalated to the License Management Group.

Step 5A — Automated Provisioning

The backend communicates with:

- Adobe APIs
- BlueBeam APIs
- Microsoft O365 APIs

to:

- Assign licenses
- Validate provisioning success
- Capture provisioning metadata

Step 5B — Human Escalation Workflow

If escalation is required:

- Freshservice ticket is updated
- AI-generated context notes are attached
- Ticket is assigned to License Management Group

Assignment strategies include:

- Round Robin
- First Available
- Priority-Based Assignment

Step 6 — Ticket Resolution

After successful provisioning:

- Freshservice ticket status changes to “Resolved”
- AI actions and notes are appended
- Resolution metadata is stored

Step 7 — User Notification

The user receives confirmation through:

- Google Chat Bot

- STACK Dashboard

Example:

“Your Adobe license has been successfully activated.”

7. AI Decision Logic

Confidence Score	Action
≥ 85%	Fully automated provisioning
60–84%	Provision + review flag
< 60%	Human escalation

8. Logging & Audit Trail

At every workflow stage, logs are written to Elasticsearch.

Logged events include:

- User requests
- AI decisions
- API calls
- Ticket updates
- Human interventions
- Resolution actions
- Error handling events

This enables:

- Audit compliance
- Operational monitoring
- AI traceability
- Incident analysis

9. Security & Governance

Authentication

- OAuth 2.0
- JWT Token Validation

Authorization

- Role-based provisioning controls
- License entitlement validation

Auditability

- Full activity logging
- AI decision traceability

Compliance

- SOP-based automation enforcement
- Controlled escalation workflows

10. Benefits

Benefit	Impact
Faster ticket resolution	Reduced turnaround time
Reduced manual effort	Improved IT efficiency
AI-driven automation	Scalable operations
Intelligent escalation	Better decision accuracy
Centralized audit logging	Improved governance
Improved user experience	Faster employee onboarding/productivity

11. Future Enhancements

Potential roadmap enhancements include:

- Multi-license bundle provisioning
- Predictive license demand analytics

- AI-based cost optimization
- Auto-reclaim unused licenses
- Teams/Slack integration
- Approval workflow automation
- Policy-aware provisioning

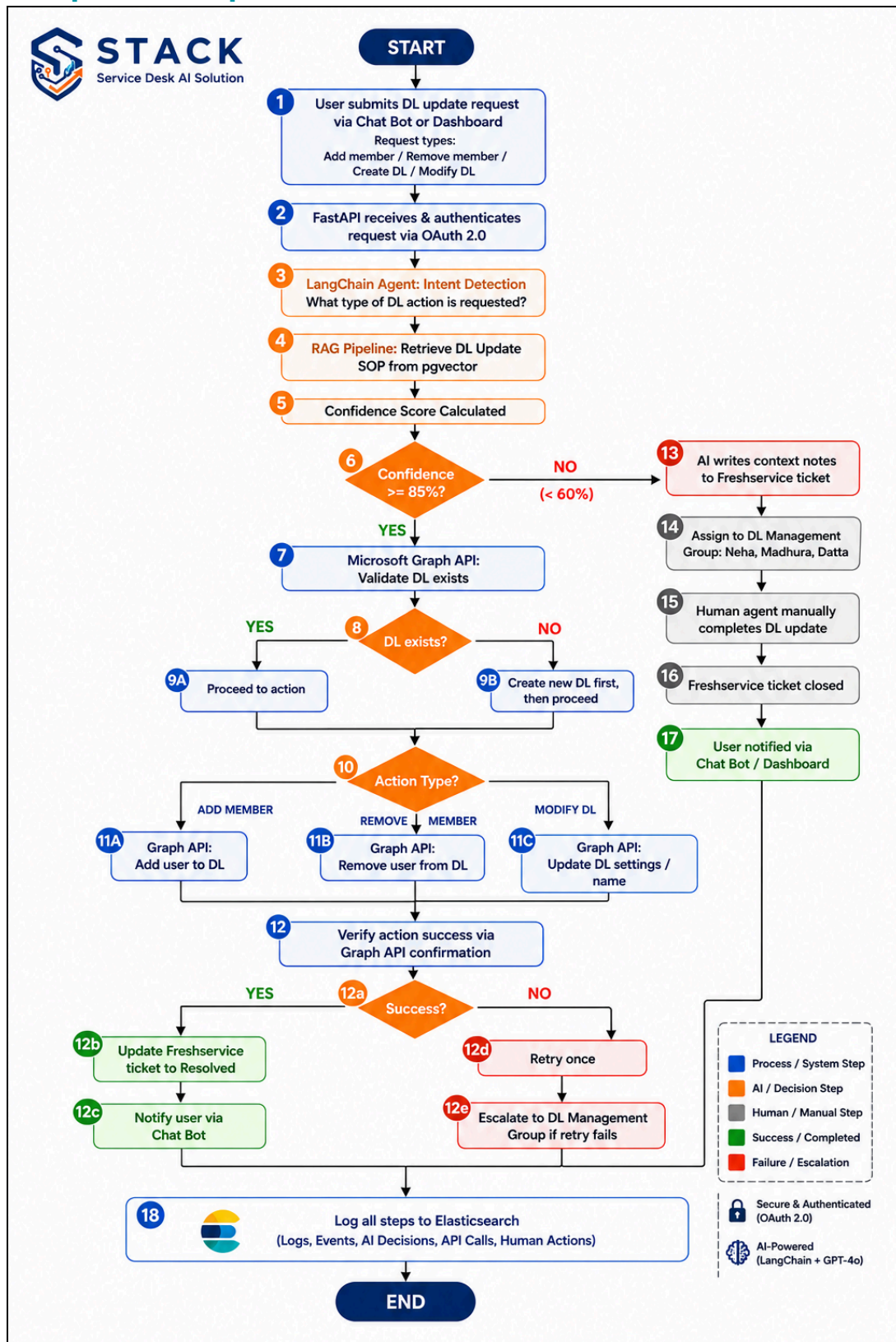
12. Conclusion

The STACK Service Desk AI Solution modernizes enterprise license management by combining:

- Conversational AI
- Intelligent workflow orchestration
- Automated provisioning
- Human-in-the-loop governance

The platform significantly reduces operational overhead while improving speed, consistency, and user satisfaction across IT support operations.

9.3 DL updates requests O365



1. Overview

The **STACK Service Desk AI Solution** is an AI-powered IT Service Desk automation platform developed to streamline enterprise support operations using Generative AI, workflow orchestration, and intelligent automation.

This workflow specifically focuses on **Distribution List (DL) Update Automation** within the Microsoft 365 ecosystem. The solution automates common DL-related requests such as:

- Adding members to Distribution Lists
- Removing members from Distribution Lists
- Creating new Distribution Lists
- Modifying Distribution List settings

The system leverages:

- FastAPI-based backend services
- LangChain AI orchestration
- Azure OpenAI GPT-4o
- Microsoft Graph API
- Freshservice ITSM integration
- Elasticsearch logging and audit tracking

The primary objective is to reduce manual IT workload, improve response time, standardize execution using SOPs, and increase operational efficiency through AI-driven automation.

2. Business Objective

The DL Update Automation workflow addresses common operational challenges faced by enterprise IT support teams:

Existing Challenges

- High volume of repetitive DL update requests
- Manual validation and execution effort
- Delayed turnaround times
- Human dependency for simple tasks
- Lack of standardized execution
- Inconsistent audit trails
- Increased operational overhead

Solution Goals

The STACK AI Solution aims to:

- Automate repetitive Distribution List operations
- Reduce ticket resolution time
- Improve user experience through self-service channels
- Standardize actions using SOP-based automation
- Enable intelligent decision-making using AI confidence scoring
- Escalate only low-confidence requests to human agents
- Maintain complete observability and audit logging

3. High-Level Workflow Summary

The automation workflow follows a structured AI-driven execution model:

1. User submits a Distribution List request via Chat Bot or Dashboard
2. FastAPI authenticates and validates the request
3. LangChain AI Agent identifies the request intent
4. RAG pipeline retrieves SOP guidance from vector storage
5. AI calculates confidence score
6. High-confidence requests are auto-executed using Microsoft Graph API
7. Low-confidence requests are escalated to human support teams
8. Ticket status and notifications are updated automatically
9. All actions are logged into Elasticsearch

4. System Components

4.1 User Interface Layer - Google Chat Bot / Web Dashboard

The user interface layer acts as the entry point for service requests.

Capabilities

- User self-service interaction
- Natural language request submission
- Real-time request tracking
- Automated notifications
- Secure authentication support

Example Requests

- "Add John to Finance DL"
- "Remove user from HR mailing list"
- "Create new project distribution list"
- "Update DL owner and permissions"

4.2 API Gateway & Backend Layer - FastAPI Backend

The FastAPI backend acts as the orchestration and integration layer.

Responsibilities

- OAuth 2.0 authentication validation
- Request parsing and normalization
- Ticket context processing
- AI workflow invocation
- Microsoft Graph API communication
- Freshservice ticket updates
- Error handling and retries

Key Benefits

- Lightweight and scalable architecture
 - High-performance asynchronous processing
 - REST API integration support
 - Enterprise-grade extensibility
-

4.3 AI & Intelligence Layer - LangChain AI Agent

The LangChain Agent serves as the intelligent decision engine.

(i) AI Responsibilities

- Intent detection
- Request classification
- SOP retrieval
- Confidence scoring
- Decision making
- Action recommendation

(ii) Supported Intent Types

- Add Member
- Remove Member
- Create DL
- Modify DL

(iii) Azure OpenAI GPT-4o

GPT-4o is used for:

- Natural language understanding
- Context interpretation
- SOP reasoning
- Intelligent workflow decisions
- Ticket summarization

(iv) RAG Pipeline with pgvector

The Retrieval-Augmented Generation (RAG) pipeline enables the AI engine to retrieve organizational SOPs and operational knowledge.

Components

- SOP documents
- Operational runbooks
- Historical ticket references
- Vector embeddings stored in pgvector

Benefits

- Context-aware automation
 - Reduced hallucination risk
 - SOP-aligned execution
 - Consistent operational standards
-

4.4 Automation & Integration Layer - Microsoft Graph API

The Microsoft Graph API is used to perform automated DL operations within Microsoft 365.

(i) Automated Actions

Distribution List Validation

- Verify DL existence
- Validate user identities
- Retrieve DL metadata

Distribution List Operations

- Add member to DL
- Remove member from DL
- Create new DL
- Update DL settings
- Modify DL ownership

(ii) Validation Mechanism

After execution, Graph API confirmation responses are validated to ensure action success.

4.5 ITSM Integration Layer - Freshservice ITSM

Freshservice acts as the centralized ticketing and workflow management platform.

(i) Responsibilities

- Ticket creation
- Ticket status updates
- AI context note storage
- Human escalation management
- Resolution tracking
- Audit references

(ii) Automated Ticket Updates

- In Progress
 - Auto-Resolved
 - Escalated
 - Closed
-

4.6 Human Escalation Layer - DL Management Group

Low-confidence or failed automation scenarios are escalated to the DL Management Group.

(i) Support Team Members

- Neha
- Madhura
- Datta

(ii) Escalation Triggers

- AI confidence score below threshold
- API execution failure
- Retry failure
- Invalid request structure
- Missing approvals
- Permission conflicts

(iii) Human Agent Responsibilities

- Review AI-generated context
- Manually execute DL updates
- Validate compliance requirements
- Resolve complex scenarios
- Close Freshservice tickets

5. Detailed Workflow Explanation

Step 1 – User Request Submission

The process begins when an end user submits a Distribution List request using either:

- Google Chat Bot
- Web Dashboard

Supported Request Types

- Add Member
- Remove Member
- Create DL
- Modify DL

The request is captured in natural language format.

Step 2 – Authentication & Validation

The request is forwarded to the FastAPI backend.

Validation Activities

- OAuth 2.0 authentication
- User identity validation
- Request schema validation
- Authorization checks

Only authenticated and authorized requests proceed further.

Step 3 – AI Intent Detection

The LangChain AI Agent analyzes the request.

AI Analysis

- Identify requested DL action
- Extract entity information
- Determine operation category
- Validate request completeness

Example:

Request:

“Add Rahul to Finance Distribution List”

AI Interpretation:

- Action Type: Add Member
 - Target DL: Finance DL
 - User: Rahul
-

Step 4 – SOP Retrieval Using RAG

The AI engine retrieves relevant SOPs and operational procedures from pgvector using the RAG pipeline.

Retrieved Information

- DL creation procedures
- Naming standards
- Permission requirements
- Compliance rules
- Execution workflows

This ensures the AI follows enterprise-approved processes.

Step 5 – Confidence Score Calculation

The AI system calculates a confidence score based on:

- Intent clarity
- SOP match quality
- Request completeness
- Entity extraction accuracy
- Historical similarity

Confidence Thresholds

Confidence Score	Action
>= 85%	Fully automated execution
60% – 84%	Assisted automation / review
< 60%	Escalation to human agent

Step 6 – Distribution List Validation

Before execution, the system validates whether the requested Distribution List exists.

Validation Checks

- DL availability
- Naming conflicts
- Existing ownership
- Membership status

Decision Outcomes

DL Exists

The system proceeds with the requested action.

DL Does Not Exist

The system creates the DL first and then continues the workflow.

Step 7 – Automated DL Actions

Based on the identified intent, the system performs the requested action using Microsoft Graph API.

Add Member

- User added to Distribution List
- Membership validated

Remove Member

- User removed from Distribution List
- Membership updated

Modify DL

- Update DL settings
 - Modify ownership
 - Rename Distribution List
 - Update configuration parameters
-

Step 8 – Verification & Confirmation

After execution, the system validates the API response.

Validation Includes

- API response status
 - Membership confirmation
 - DL configuration validation
 - Transaction completion status
-

Step 9 – Success Handling

If the operation succeeds:

Automated Actions

- Freshservice ticket updated to “Resolved”
- AI execution notes attached
- Resolution summary generated
- User notified through Chat Bot or Dashboard

User Notification Example

“Your Distribution List request has been successfully completed.”

Step 10 – Failure Handling & Retry Logic

If the action fails:

Retry Process

- System retries once automatically
- Retry logs captured in Elasticsearch

Escalation Trigger

If retry fails, the ticket is escalated to the DL Management Group.

Step 11 – Human Escalation Workflow

For low-confidence or failed automation requests:

AI-Assisted Escalation

The AI system automatically:

- Generates context notes
- Attaches SOP references
- Summarizes attempted actions
- Adds failure details
- Assigns the ticket to support team

Human Resolution

The support team manually performs the required updates and closes the ticket.

6. Elasticsearch Logging & Observability

Every workflow step generates logs for monitoring and audit purposes.

Logged Events

- User requests
- Authentication events
- AI decisions
- Confidence scores
- Graph API calls
- Ticket updates
- Human interventions
- Errors and retries

Benefits

- Full audit trail
 - Troubleshooting support
 - Operational analytics
 - Compliance reporting
 - AI performance analysis
-

7. Security Architecture

The solution follows enterprise-grade security standards.

Security Controls

- **OAuth 2.0 Authentication**
Ensures only authenticated users can submit requests.
 - **Role-Based Access Control (RBAC)**
Restricts execution based on user permissions.
 - **Secure API Communication**
All integrations use encrypted HTTPS communication.
 - **Audit Logging**
Every action is traceable and logged.
 - **AI Governance**
SOP-driven AI execution minimizes unauthorized actions.
-

8. Business Benefits

Operational Benefits

- Faster ticket resolution
- Reduced manual effort
- Increased automation coverage
- Improved SLA compliance
- Standardized execution

User Experience Benefits

- Faster response times
- Self-service enablement
- Real-time notifications
- Reduced dependency on support teams

IT Support Benefits

- Reduced repetitive workload
- Better focus on critical issues
- AI-assisted decision making
- Improved operational visibility

9. Future Enhancements

Potential future improvements include:

- Multi-language support
 - Predictive ticket categorization
 - Automated approval workflows
 - Adaptive confidence thresholds
 - AI-based anomaly detection
 - Integration with Teams and Slack
 - Advanced analytics dashboards
 - Autonomous remediation workflows
-

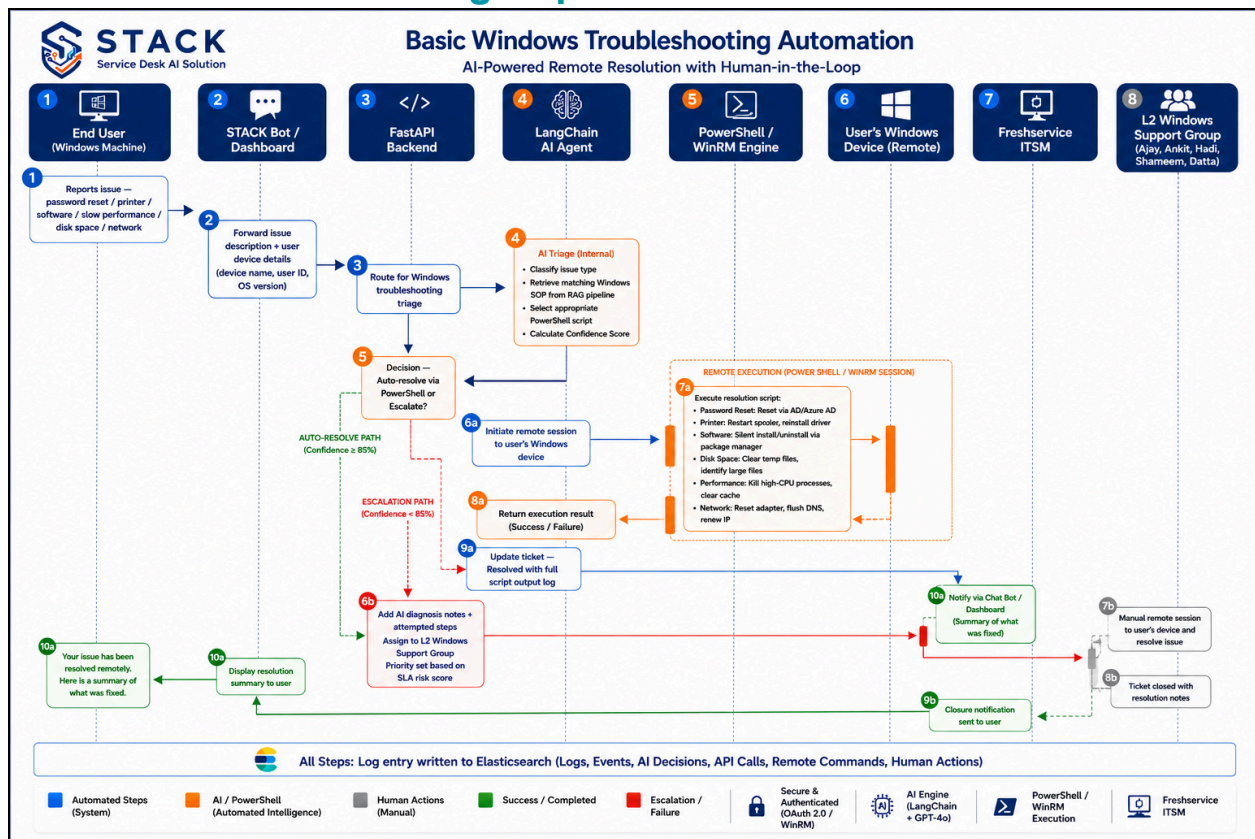
10. Conclusion

The STACK Service Desk AI Solution provides an intelligent, scalable, and enterprise-ready automation framework for Microsoft 365 Distribution List management.

By combining Generative AI, LangChain orchestration, RAG-based SOP retrieval, Microsoft Graph API integration, and Freshservice ITSM workflows, the solution significantly improves operational efficiency while maintaining governance, security, and auditability.

The DL Update Automation workflow demonstrates how AI-powered IT operations can reduce manual intervention, accelerate ticket resolution, and enhance the overall employee support experience.

9.4 Basic troubleshooting requests Windows



1. Overview

The **STACK Service Desk AI Solution** provides an AI-powered automated troubleshooting framework for resolving common Windows-related IT issues using intelligent triage, SOP-driven automation, PowerShell remoting, and Freshservice ITSM integration.

This workflow enables automatic resolution of repetitive L1 support tasks while escalating complex issues to the L2 Windows Support Team when required.

2. Objective

The objective of this workflow is to:

- Reduce manual IT support workload
- Improve Mean Time to Resolution (MTTR)
- Enable 24x7 self-service IT support
- Automate repetitive Windows troubleshooting tasks
- Improve user experience through AI-assisted resolution
- Maintain auditability and compliance through centralized logging

3. Actors / Components

Component	Description
End User	Employee reporting Windows-related issue
STACK Bot / Dashboard	User interaction layer via Chat Bot or Web Dashboard
FastAPI Backend	Core orchestration and API processing layer
LangChain AI Agent	AI decision engine powered by GPT-4o + RAG
PowerShell / WinRM Engine	Remote automation execution engine
User Windows Device	Target machine for troubleshooting actions
Freshservice ITSM	Ticket lifecycle management system
L2 Windows Support Group	Human escalation team for unresolved issues
Elasticsearch	Centralized logging and audit trail system

4. Supported Use Cases

The automation currently supports the following Windows troubleshooting categories:

- Password Reset
- Printer Issues
- Software Installation / Uninstallation
- Slow System Performance
- Disk Space Cleanup
- Network Connectivity Issues

5. Workflow Description

Step 1 — Issue Submission

The end user reports an issue through:

- STACK Chat Bot
- STACK Web Dashboard

Example issues include:

- Password reset request
- Printer not working
- Slow laptop performance
- Low disk space
- Software installation request
- Network connectivity issue

The request includes:

- Device Name
 - User ID
 - Operating System Version
 - Issue Description
-

Step 2 — Request Processing

The STACK Bot forwards the request to the FastAPI backend.

The backend:

- Validates authentication
 - Verifies OAuth/JWT token
 - Creates or updates Freshservice ticket
 - Normalizes issue metadata
-

Step 3 — AI-Based Triage

The FastAPI backend routes the request to the LangChain AI Agent.

The AI engine performs:

Intent Classification

Identifies the issue category:

- Password
- Printer
- Software
- Performance
- Disk Space
- Network

SOP Retrieval (RAG Pipeline)

The AI retrieves the most relevant SOP from the enterprise knowledge base using:

- pgvector embeddings
- Semantic search
- Historical incident patterns

Script Selection

Based on the identified issue, the AI selects the appropriate PowerShell remediation script.

Confidence Scoring

The AI calculates a confidence score to determine whether automation is safe.

AI Decision Logic

Confidence Score	Action
≥ 85%	Automatic remediation
< 85%	Escalate to L2 support

Step 4 — Remote Session Initialization

FastAPI initiates a secure remote PowerShell/WinRM session to the user's Windows machine.

Security controls include:

- Encrypted WinRM communication
 - RBAC validation
 - Session logging
 - OAuth-secured execution
-

Step 5 — PowerShell Script Execution

The automation engine executes issue-specific remediation scripts.

Password Reset

- Reset AD/Azure AD credentials
- Unlock user account

Printer Troubleshooting

- Restart Print Spooler
- Reinstall printer driver
- Re-map printer queue

Software Support

- Silent install/uninstall
- Validate package dependencies

Disk Space Cleanup

- Clear temp/cache files
- Remove stale logs
- Identify large files

Performance Optimization

- Kill high CPU processes
- Clear cache
- Restart impacted services

Network Troubleshooting

- Reset network adapter
 - Flush DNS
 - Renew IP configuration
-

Step 6 — Execution Result Validation

The Windows device returns execution status to the PowerShell engine.

Possible results:

- Success
- Partial Success
- Failure

Execution logs are captured and forwarded to FastAPI.

Step 7 — Ticket Update

FastAPI updates the Freshservice ticket with:

- Resolution status
- Script output logs
- Actions performed
- Timestamp
- AI decision metadata

Ticket status is updated to:

Resolved

Step 8 — User Notification

The user receives a notification through:

- STACK Chat Bot
- Web Dashboard
- Email notification (optional)

Example response:

“Your issue has been resolved remotely. A summary of the remediation steps has been provided.”

6. Escalation Workflow

If AI confidence is below threshold or automation fails:

AI Escalation Actions

FastAPI performs the following:

- Adds AI diagnosis notes
 - Attaches attempted remediation logs
 - Adds SOP references
 - Calculates SLA risk score
 - Assigns ticket priority
-

7. L2 Support Assignment

The ticket is assigned to the:

L2 Windows Support Group

- Ajay
 - Ankit
 - Hadi
 - Shameem
 - Datta
-

8. Manual Resolution

The L2 engineer:

- Opens remote session
- Performs advanced troubleshooting
- Resolves issue manually
- Updates ticket with resolution notes

Freshservice ticket is then closed.

9. Logging & Observability

All workflow events are logged into Elasticsearch.

Logged Events Include

- User requests
- AI decisions
- PowerShell execution logs
- API calls
- Authentication events
- Ticket updates
- Human actions
- Error traces

10. Security & Compliance

The solution includes enterprise-grade security controls:

- OAuth 2.0 Authentication
- JWT Authorization
- Secure WinRM communication
- RBAC enforcement
- Audit logging
- Encrypted API communication
- Session tracking

11. Key Benefits

Benefit	Impact
Reduced MTTR	Faster issue resolution
Lower L1 workload	Reduced manual repetitive tasks
24x7 Availability	Continuous AI support
Consistent troubleshooting	SOP-driven automation
Better user experience	Faster response times
Full auditability	Centralized logging & tracking
Scalable operations	Supports enterprise growth

12. Technology Stack

Layer	Technology
Frontend	React.js Dashboard + Google Chat
Backend	FastAPI (Python)
AI Layer	Azure OpenAI GPT-4o + LangChain
Knowledge Retrieval	RAG + pgvector
Automation	PowerShell + WinRM
ITSM	Freshservice
Logging	Elasticsearch + Kibana
Monitoring	Grafana + Prometheus
Containerization	Docker

13. Conclusion

The STACK Service Desk AI Solution modernizes Windows IT support by combining AI-powered triage, intelligent SOP retrieval, and remote automation into a scalable enterprise support platform.

By automating repetitive troubleshooting activities and integrating seamlessly with Freshservice and Microsoft technologies, the platform significantly improves operational efficiency, resolution speed, and end-user satisfaction while maintaining strong governance and auditability.

10. Team Roles & Responsibilities

10.1 Nikhil — Lead Developer

Nikhil — Lead Developer
• Overall system architecture and all technical decisions
• FastAPI backend — all APIs, business logic, middleware
• AI & Intelligence Core — LangChain agents, RAG pipeline, Azure OpenAI integration, SOP ingestion & embeddings
• Automation Engine — PowerShell/WinRM scripts for Windows troubleshooting
• Microsoft Graph API integration — SharePoint, O365, DL management
• License API integrations — Adobe, BlueBeam, O365
• Freshservice REST API integration — ticket ingestion, status updates, closures
• React.js Web Dashboard — UI, dashboards, reporting, logs viewer, admin controls
• PostgreSQL database design and management
• Redis, Elasticsearch, Kibana setup and configuration
• Docker, Nginx, on-premise infrastructure setup
• Grafana + Prometheus monitoring
• Gitea + Drone CI pipeline setup
• SOP library creation and maintenance
• API contract documentation (Swagger) for Priyanka's integration
• Final end-to-end integration and system testing

10.2 Priyanka — Chatbot Developer

Priyanka — Chatbot Developer

- Google Chat Bot — full design, development, and deployment
- Google Chat App configuration in Google Workspace Admin Console
- Bot conversation flows for all 4 use cases (SharePoint, License, DL, Troubleshooting)
- Natural language UX within chat, powered by shared LangChain + GPT-4o backend
- Google OAuth / Service Account setup for bot authentication
- Bot webhook integration with Nikhil's FastAPI backend
- User-facing conversation design — greetings, ticket creation, status checking, resolution confirmation, escalation messages
- Bot testing with real Google Workspace users
- Bot conversation flow documentation
- Google Chat Bot deployment and post-launch maintenance

10.3 Collaboration Model

Daily Sync:

- Frequency: Every day
- Duration: 15 minutes standup
- Format: What did you do yesterday? What are you doing today? Any blockers?
- Platform: Google Chat or Google Meet

Git Workflow:

- Single Git repository with branch convention: main, dev, nikhil/feature-x, priyanka/chatbot-x
- Nikhil owns all merges into main and dev branches
- Priyanka works exclusively on chatbot/ prefixed branches

Key Integration Handoff:

Critical Dependency: Nikhil to deliver the FastAPI webhook endpoint spec (/api/v1/chat/message) to Priyanka by June 1. This enables Priyanka to begin bot integration without waiting for the full backend to be complete. All bot requests and responses flow through this single endpoint.

11. Project Plan — Target: June 10, 2025

Phase 1 — Foundation (May 27 – May 30)

Task	Owner	Deadline
Project repository setup on Gitea	Nikhil	May 27
PostgreSQL + Redis + Docker Compose setup	Nikhil	May 28
FastAPI project scaffolding + base auth (OAuth/JWT)	Nikhil	May 29
Freshservice API connection & ticket ingestion	Nikhil	May 30
Google Chat App registration in Workspace Admin	Priyanka	May 28
Bot project scaffolding + webhook listener setup	Priyanka	May 30

Phase 2 — Core AI & Automation (May 31 – June 4)

Task	Owner	Deadline
Azure OpenAI + LangChain setup and configuration	Nikhil	May 31
SOP ingestion + pgvector embeddings pipeline	Nikhil	June 1
RAG pipeline + LangChain agent (intent detection, confidence scoring)	Nikhil	June 2
/api/v1/chat/message endpoint ready for Priyanka	Nikhil	June 1
SharePoint automation via Microsoft Graph API	Nikhil	June 3
License automation — Adobe, BlueBeam, O365	Nikhil	June 3
DL update automation via Microsoft Graph API	Nikhil	June 4
PowerShell / WinRM Windows troubleshooting scripts	Nikhil	June 4
Bot conversation flows — ticket creation & status check	Priyanka	June 2
Bot integration with /api/v1/chat/message endpoint	Priyanka	June 3
Bot flows — resolution updates & human escalation messages	Priyanka	June 4

Phase 3 — Dashboard, Logs & Reporting (June 5 – June 7)

Task	Owner	Deadline
Elasticsearch + Kibana log pipeline setup	Nikhil	June 5
React dashboard — live ticket queue & logs viewer	Nikhil	June 5
React dashboard — reporting, analytics charts, ROI dashboard	Nikhil	June 6
React dashboard — admin controls, escalation config, thresholds	Nikhil	June 6
Grafana + Prometheus monitoring setup	Nikhil	June 7
Bot end-to-end testing across all 4 use cases	Priyanka	June 5–6
Bot UX refinements based on testing feedback	Priyanka	June 7

Phase 4 — Integration Testing & Deployment (June 8 – June 10)

Task	Owner	Deadline
Full end-to-end integration testing (all use cases)	Nikhil + Priyanka	June 8
Bug fixes & edge case handling	Nikhil + Priyanka	June 9
On-premise Docker deployment	Nikhil	June 9
Final review & project sign-off	Nikhil + Priyanka	June 10

12. Security & Compliance

- All API communication encrypted over HTTPS via Nginx TLS termination
- OAuth 2.0 + JWT for all user authentication — no passwords stored in the system
- PowerShell/WinRM actions fully logged with complete audit trail (who, what, when, outcome)
- Azure OpenAI — all data stays within your Azure tenant; no data shared with public OpenAI
- Role-Based Access Control (RBAC) — Admins vs Agents vs Read-Only users
- All automation actions require pre-approval configuration per use case in the Admin Dashboard
- Backup and Disaster Recovery via Azure Blob Storage with configurable retention policy
- Escalation group membership and assignment modes configurable only by Admin-role users

13. Glossary

Term	Definition
ITSM	IT Service Management — the system used to log, track, and manage IT support tickets
Freshservice	The cloud-based ITSM tool used in this project for ticket management
RAG	Retrieval-Augmented Generation — AI technique to answer questions using your own documents as a knowledge base
SOP	Standard Operating Procedure — step-by-step internal guide for resolving a specific ticket type
LangChain	Python framework for building AI agent workflows and RAG pipelines
LangChain Agent	An AI component that can reason, make decisions, and take actions autonomously
pgvector	A PostgreSQL extension that enables storing and searching AI embeddings (vectorised text)
WinRM	Windows Remote Management — allows executing PowerShell commands on remote Windows machines
Graph API	Microsoft's unified REST API for interacting with O365, SharePoint, Teams, and more
FastAPI	A modern Python web framework used for the backend API engine
DL	Distribution List — an O365 email group that delivers messages to multiple recipients
Confidence Score	A 0-100% score the AI calculates before auto-resolving a ticket, based on intent clarity, SOP match, history, and input completeness
SLA	Service Level Agreement — the agreed maximum time to resolve a ticket
ROI	Return on Investment — financial measure of the value generated by the system vs its cost
Celery	A Python task queue library used to run background automation jobs asynchronously
Redis	An in-memory data store used as a cache and as Celery's task broker
Elasticsearch	A distributed search and analytics engine used for log storage and full-text search
Kibana	A visualization tool for Elasticsearch data, used for log dashboards